PROJET D'IHM 2026 · ÉTAPE 2

Corpus pédagogique de l'agent

Six notions, trois langages, un jeu de questions gelé avant toute mesure — version **corpus-v1**, généré le 2026-10-27

1. Ce que cette étape corrige

Le corpus de démonstration de la version précédente tenait en onze fragments, répartis **5 Python / 3 Java / 3 C++**, et mêlés à des corrigés d'exercices. Un agent qui s'appuie là-dessus ne peut pas être équitable entre les trois parcours : il répond mieux en Python, et il peut citer une correction que l'enseignant n'a pas encore donnée. L'étape 2 remplace ce jeu par une **grille** : chaque notion enseignée existe dans les trois langages, dans le même nombre de fragments, et aucun corrigé n'y entre.

Avant (v0.3.3)	Après (corpus v1)
11 fragments	**36 fragments** — 18 cours
5 Python · 3 Java · 3 C++	**12 Python · 12 Java · 12 C++**
1 notion couverte (fonctions)	**6 notions** × 3 langages
Corrigés mêlés au cours	Aucun : seuls les blocs `role = COURS` sont admis
Aucun jeu de questions figé	**60 questions gelées** avant la première mesure

> **La règle qui gouverne tout le reste**
> Un fragment du corpus est ce que l'agent peut **citer**. Ce qu'il ne peut pas citer, il ne le dira pas : c'est ce qui rend vraie la promesse « l'agent ne diffuse pas de corrigé » autrement qu'en le demandant au modèle.

2. La grille — 6 notions × 3 langages

Notion	Python	Java	C++
Variables et types	Variables et types en Python 2 fragments	Variables et types en Java 2 fragments	Variables et types en C++ 2 fragments
Conditions	Conditions en Python 2 fragments	Conditions en Java 2 fragments	Conditions en C++ 2 fragments
Boucles	Boucles en Python 2 fragments	Boucles en Java 2 fragments	Boucles en C++ 2 fragments
Fonctions et valeur de retour	Fonctions et valeur de retour en Python 2 fragments	Méthodes et valeur de retour en Java 2 fragments	Fonctions, retour et surcharge en C++ 2 fragments
Collections	Listes et dictionnaires en Python 2 fragments	Collections en Java 2 fragments	Conteneurs STL en C++ 2 fragments
Erreurs et exceptions	Erreurs et exceptions en Python 2 fragments	Exceptions en Java 2 fragments	Exceptions en C++ 2 fragments

Chaque cours porte deux blocs : un **TEXTE** qui explique la notion, un **CODE** qui la montre dans le langage concerné. Les titres restent descriptifs (« Listes et dictionnaires en Python » pour la notion Collections) : c'est le contenu qui définit la notion, pas un préfixe.

3. Le nettoyage — quatre passes, pas une

Passe	Règle	Pourquoi cet ordre
1. Normalisation	Unicode NFC, espaces insécables, apostrophes typographiques, `\n` littéraux	Deux écritures du même texte doivent se reconnaître avant d'être comparées.
2. Filtre de rôle	Seuls les blocs `role = COURS` sont admis	Un corrigé cité est une correction donnée : exclusion par construction, pas par consigne.
3. Longueur minimale	150 caractères pour un TEXTE, 40 pour un CODE	Un fragment trop court n'a pas de contexte citable.
4. Dédoublonnage	Empreinte exacte, puis proximité de Jaccard ≥ 0,85	Deux fragments qui redisent la même chose font citer deux fois la même notion.

Résultat des passes sur la source : **0** rôle(s) non conforme(s) retiré(s) · **0** bloc trop court · **0** doublon exact · **0** quasi-doublon. Le nettoyage n'a donc rien retiré ici — parce que la source a été écrite pour la grille. La valeur de ces passes est de **tenir** quand la source grandira, pas de produire un chiffre flatteur.

Mesures de forme : 10751 caractères au total, **299** par fragment en moyenne (min 145, max 437), sous la limite de découpage de 600 caractères — aucun fragment n'est coupé au milieu d'un mot.

> **Un défaut trouvé par l'exécution, pas par la relecture**
>
> La normalisation écrasait les espaces multiples. Sur un bloc de code, cela détruisait l'indentation : le programme affiché devenait faux, alors que le texte, lui, restait lisible. Le défaut ne se voyait pas dans les compteurs — il a été vu en relisant un fragment de code exporté. La règle « réduire les espaces » ne s'applique plus qu'aux blocs TEXTE.

4. Le jeu de 60 questions, gelé avant la mesure

Un jeu de test écrit après les réglages ne mesure plus rien : il mesure le réglage. Les 60 questions ont donc été écrites, vérifiées puis **figées** avant la première exécution de l'évaluation. L'empreinte SHA-256 du fichier est enregistrée dans `ia/corpus/MANIFESTE.json`, avec la date de gel (**2026-10-27**) — une régénération ultérieure du corpus ne déplace pas cette date si les questions n'ont pas changé.

Notion	Python	Java	C++	Transversale	Total
Variables et types	3	3	3	1	10
Conditions	3	3	3	1	10
Boucles	3	3	3	1	10
Fonctions et valeur de retour	3	3	3	1	10
Collections	3	3	3	1	10
Erreurs et exceptions	3	3	3	1	10

Chaque question porte un témoin

Le témoin est le mot que doit contenir le fragment cité pour que la réponse soit juste. Sans lui, un agent qui cite la bonne section mais la mauvaise phrase passerait pour bon.

Question	Énoncé	Langage	Témoin attendu
BOUCLE-c1	*« Le for sur intervalle en C++ modifie-t-il le conteneur quand on écrit for (int n : notes) ? »*	cpp	copie
FONC-p2	*« Que renvoie une fonction Python sans return ? »*	python	None
COLL-j1	*« Quelle est la différence entre length et size() en Java ? »*	java	length
ERREUR-c2	*« Quelle exception lève vector::at avec un indice invalide ? »*	cpp	out_of_range

> **Le jeu a refusé de se geler, une fois**
> La question « comment passer un vecteur à une fonction C++ sans le copier » avait pour témoin *référence*. Le contrôle a répondu que ce mot n'existait pas dans le cours C++ attendu : lacune réelle du corpus, et non erreur du jeu. Le cours a été complété, puis le jeu gelé. Un jeu de test qui ne peut pas échouer ne vérifie rien.

5. Ce que la mesure donne

Évaluation exécutée après le gel, sur PostgreSQL 17 + pgvector, seuil de similitude 0,05, filtre de langage actif — 60 questions, plus 4 questions hors domaine.

Mesure	Avec filtre de langage	Sans filtre
top-1 : la bonne section sort en premier	**73 % (44/60)**	63 % (38/60)
top-5 : la bonne section est dans les cinq premiers	**92 % (55/60)**	85 % (51/60)
témoin présent dans le fragment cité en premier	**77 % (46/60)**	—
témoin présent dans l'un des cinq fragments cités	**87 % (52/60)**	—

Les deux dernières lignes sont les plus sévères : elles regardent le **contenu** cité, pas seulement sa section. Le filtre de langage apporte **10 points** de top-1 : il reste nécessaire, et il ne suffit pas.

6. Ce que ce corpus ne prouve pas

- **La taille.** 36 fragments, c'est encore petit : à ce volume, une recherche par mots-clés réussit souvent. Ces taux valident la **chaîne** (découpage, vecteurs, index HNSW, recherche hybride), pas la qualité d'un agent en production.
- **La sémantique des vecteurs.** Les vecteurs sont des vecteurs de démonstration (plissement de hachage, hors ligne) : ils ne séparent pas finement le domaine du hors-domaine. C'est précisément pourquoi le refus ne repose jamais sur le seul seuil, mais aussi sur le prompt système et le filtre de sortie.
- **Le niveau pédagogique.** Tous les cours sont rattachés au rang 1 de la base de démonstration : la répartition par niveau est une **décision d'enseignant**, pas une décision de corpus.
- **Le modèle réel.** `ModeleSimule` cite, il ne rédige pas. Le corpus est prêt pour un modèle distant ; l'emplacement est en place, la clé d'API ne l'est pas.

7. Validation

Élément	État
Corpus versionné	`corpus-v1` · source `e98930314f49af01…`
Grille	18 cours · 36 fragments · 12 par langage
Jeu de questions	60 questions · gelé le 2026-10-27 · empreinte `a8f4fcc1dcfb1d0c…`
Base PostgreSQL	`db/05-corpus-v1.sql` exécuté deux fois : 18 cours, 36 blocs, aucun doublon
Application	corpus embarqué = export de la base ; `CorpusTest` (4 tests) vérifie l'équilibre 12/12/12
Suite de tests	**74 tests**, dont 29 adversariaux — verts
Décision	Corpus v1 tenu : passage à l'étape 3 (indexation et recherche) confirmé